FsVoice: A Platform for Building Voice-First Conversational Applications

TODO Author Name
TODO Affiliation
TODO@email

Abstract

FsVoice is an open-source F#/.NET platform for building voice-first conversational applications. Rather than treating speech as a microphone and speaker wrapped around a text-chat loop, FsVoice models realtime conversation as an event-driven, multi-agent orchestration that can be embedded in mobile, web, desktop, command-line, or telephony hosts. The platform provides typed realtime event contracts, host adapters, plugin-defined behavior, tool and context provider interfaces, an Oracle pattern for delegating complex language tasks to stronger reasoning models, and optional retrieval components for applications that need private or domain-specific knowledge. We demonstrate FsVoice through Speak2Docs, a mobile application for conversing with selected documents. Document question answering is one application of the platform: the same architecture supports conversational interfaces that coordinate speech, application state, domain tools, and natural-language reasoning. This demo paper presents the platform architecture, deployment topologies, extensibility model, Speak2Docs reference workflow, and an evaluation plan covering conversational latency, task completion, and document-QA retrieval quality.

1 Introduction

Realtime voice models make spoken interaction with language systems feel increasingly natural, but building useful conversational applications remains an engineering problem. A production voice application must track streaming events, handle interruptions, call tools, preserve application state, coordinate UI updates, route complex turns to stronger models, and, in many settings, ground answers in private or domain-specific information. Treating voice as a thin audio layer around a text-centric chat interface leaves these concerns outside the main application architecture.

FsVoice addresses this gap by treating conversational applications as realtime, event-driven systems. The platform exposes typed contracts for realtime sessions, orchestration, host messages, tools, context providers, plugins, and retrieval. Applications assemble these pieces according to their domain: a mobile assistant, a browser-based product navigator, an enterprise workflow helper, a contact-center agent, or a document question-answering system (Waris, 2026b).

This paper describes FsVoice as a system demonstration for applied NLP researchers and developers building voice-first conversational applications. We use Speak2Docs as the reference application because it exercises the platform end to end: realtime speech, multi-agent orchestration, local indexing, retrieval, tool calling, and spoken responses. However, document question answering is presented as one instance of the broader platform design.

2 Platform Design

FsVoice is organized around the observation that realtime speech APIs expose a bidirectional event stream rather than a simple request-response chat interface. Once connected over WebRTC or WebSockets, the application receives continuous events from the model and can respond by sending messages that modify the conversation state, configure behavior, invoke tools, or update the host application (OpenAI, 2026a). FsVoice preserves this structure instead of flattening it into a text-only abstraction.

Figure 1 summarizes the platform flow.

2.1 Typed Realtime Orchestration

The platform provides typed session contracts that separate host-specific concerns from conversational orchestration. A mobile, browser, command-line, or server host can provide audio

TODO: Insert platform diagram from `imgs/platform.png` or a paper-specific redrawing.

Figure 1: FsVoice applications assemble a host shell, realtime voice transport, typed orchestration, model clients, plugins, tools, context providers, and optional retrieval components. Speak2Docs is one assembly of these platform pieces.

permissions, UI state, storage, and settings, while the orchestration controls realtime events, agent messages, tool calls, and model interactions. This makes conversational behavior portable across hosts rather than embedded directly inside a single application shell.

FsVoice builds on a multi-agent pattern in which agents communicate over an event bus. In the Speak2Docs assembly, a Voice agent owns the realtime voice session, an Oracle agent handles complex language and tool-use tasks, and a Host agent synchronizes orchestration events with the application UI. Other conversational applications can reuse the same structure with different host messages, tools, prompts, and domain context.

2.2 Voice and Oracle Agents

Low-latency realtime voice models are optimized for fluid turn-taking. They are not always the best component for longer reasoning, domain-grounded answers, or multi-step tool use. FsVoice therefore supports a Voice + Oracle pattern. The Voice agent carries the spoken interaction and can use natural conversational fillers while waiting for slower operations. The Oracle agent maintains a persistent connection to stronger text-oriented models through the Responses API over WebSockets (OpenAI, 2026b). It can call tools, search context, consult a blackboard, and return a response that the Voice agent speaks.

This division is an NLP system design choice: conversational fluency, instruction following, reasoning, retrieval, and tool use are assigned to components whose latency and capability profiles match the task.

2.3 Plugins, Tools, and Context

FsVoice applications are configured through plugins and host overrides. Plugins define behavior profiles, prompt templates, model-role defaults, retrieval settings, voice replacements, optional settings fields, context providers, and tool providers. Tools expose domain actions to the orchestration, while context providers expose private knowledge, application state, source inventories, session memory, or external APIs.

For applications that need document or knowledge retrieval, FsVoice includes context-backed QA contracts, a runtime, source tools, durable memory interfaces, and FsColbert-backed late-interaction retrieval (Khattab and Zaharia, 2020; Waris, 2026a). Search can combine keyword and semantic evidence using reciprocal rank fusion (Cormack et al., 2009). These retrieval components are optional platform modules rather than the whole platform.

3 Deployment Topologies

FsVoice supports several deployment topologies. In a direct mobile topology, the application connects to realtime voice services and runs orchestration locally. This is the pattern used by Speak2Docs. In a browser or enterprise web topology, a client can connect to an application server, while the server hosts the FsVoice orchestration and forwards realtime events. This lets spoken conversation remain synchronized with application UI updates and business workflows. In a telephony topology, FsVoice can sit on the SIP side of a call flow and bridge phone audio into a conversational orchestration, making the same platform relevant to contact centers and enterprise phone systems.

The key point is that the platform boundary is not the document-QA task. The boundary is the conversational application: speech transport, host state, event orchestration, tools, model roles, context, and policy.

4 Reference Application: Speak2Docs

Speak2Docs is the reference application used in the demo. It is a .NET MAUI mobile application for conversing with selected documents. Users can select local sources, build or import local indexes, start a realtime voice session, ask questions by voice, and receive spoken answers grounded in retrieved material.

The application exercises FsVoice's platform abstractions:

1. The host shell manages source selection, app settings, audio permissions, storage, and connection lifetime.

2. The Voice agent manages the realtime speech session and exposes the Oracle as a tool.

3. The Oracle agent performs source-grounded answer generation using model clients, retrieval, blackboard context, and configured tools.

4. The Host agent relays orchestration events back to the app for transcript logs, activity updates, and UI state.

The screencast will show source selection, realtime connection, spoken document questions, Oracle tool use, retrieval-grounded answers, and the relationship between the app and the reusable platform pieces.

4.1 Document-QA Module

Speak2Docs supports PDF, Markdown, JSON, and bundled retrieval indexes. Markdown ingestion preserves heading structure where possible. PDF processing can use hybrid document-structure parsing with page rasterization and local layout analysis. At query time, the runtime can perform query cleanup, keyword extraction, optional query expansion, lexical filtering, semantic retrieval, source balancing, and answer generation over selected evidence.

This module demonstrates how a conversational application can add domain context and grounding. A different FsVoice application could replace document retrieval with database lookup, workflow tools, calendar actions, product search, customer-service systems, or clinical decision-support resources.

5 Evaluation

The demo track requires some form of evaluation. Because the paper presents FsVoice as a platform, evaluation should cover both general conversational behavior and the Speak2Docs document-QA application.

5.1 Conversational Platform Evaluation

TODO: Measure repeated scripted conversational tasks across at least one host configuration. Report connection success, turn completion, tool-call success, interruption/reconnect behavior, and median/p90 latency from final transcript to answer start. Include tasks that exercise host-state synchronization, tool invocation, and Oracle delegation.

5.2 Document-QA Evaluation

TODO: Run the existing InsuranceQA search and answer workflows or an equivalent public document-QA query set. Report the number of queries, retrieval configuration, answer model, and metrics such as Recall@k, MRR, nDCG, answer correctness, support quality, and refusal/error rate.

5.3 Developer-Facing Evaluation

TODO: Report the concrete replacement points used by Speak2Docs: host shell, plugin, model roles, context providers, tools, retrieval policy, and storage. If possible, include a small second configuration, such as CLI or ASP.NET hosting, to demonstrate that the platform is not a single hard-coded mobile app.

6 Comparison With Existing Systems

FsVoice differs from conventional document-chat systems by making realtime speech orchestration a first-class system boundary. It differs from single-application voice demos by exposing typed contracts and plugin points for host integration, tools, context providers, model roles, retrieval policy, and runtime behavior. Compared with text-only RAG frameworks, FsVoice must coordinate audio streaming, realtime tool calls, spoken turn-taking, host UI state, and optional source grounding.

TODO: Add related systems and libraries: voice assistants, realtime agent frameworks, RAG frameworks, document-chat systems, ColBERT/FsColbert, voice UI toolkits, and spoken-dialogue systems.

7 Availability and Licensing

FsVoice source code is available at `https://github.com/fwaris/FsVoice` (Waris, 2026b). The source code is licensed under MIT. The retrieval package documents additional license information for embedded model assets. The demo submission will include a downloadable package or installation link for the Speak2Docs reference application, plus a screencast of at most 2.5 minutes.

TODO: Add final live demo or installable package link. TODO: Add video link.

8 Conclusion

FsVoice demonstrates a platform approach to conversational applications. Its central architectural move is to treat realtime voice as an event-driven

orchestration problem and to separate fluent spoken interaction from slower reasoning, tool use, and domain context. Speak2Docs shows this pattern in a document-question answering application, while the same platform pieces can be reused for other mobile, web, enterprise, and telephony conversational systems.

Limitations

FsVoice currently depends on external hosted language and realtime voice models, so availability, latency, cost, and privacy properties depend partly on those services (OpenAI, 2026a,b). The reference app requires a user-provided API key and network access for realtime voice and answer generation. Voice interaction quality can be affected by microphone quality, background noise, and self-interruption from speaker audio. The current evaluation is planned around platform latency, demo robustness, and document-QA quality; broader user studies and additional non-document applications are future work.

Ethical Considerations

FsVoice applications may send microphone audio, transcripts, prompts, tool outputs, application state, and selected domain context to external model services, depending on host configuration. Speak2Docs asks the user to acknowledge this data flow before connecting. Because conversational systems can produce unsupported or incomplete answers, the demo will expose grounding behavior and limitations rather than presenting spoken answers as authoritative. The paper will avoid private documents and will use public or appropriately licensed sources for evaluation and demonstration.

References

Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. 2009. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759.

Omar Khattab and Matei Zaharia. 2020. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48.

OpenAI. 2026a. Openai realtime api guide. https://platform.openai.com/docs/guides/realtime. Accessed 2026-06-04.

OpenAI. 2026b. Openai responses api reference. https://platform.openai.com/docs/api-reference/responses. Accessed 2026-06-04.

Faisal Waris. 2026a. Fscolbert. https://github.com/fwaris/fscolbert.

Faisal Waris. 2026b. Fsvoice. https://github.com/fwaris/FsVoice.

A Demo Script

TODO: Add the final 2.5 minute video script. The script should introduce FsVoice as the platform, then use Speak2Docs as the concrete demonstration.

B System Configuration

TODO: Add model roles, host topology, retrieval settings, source bundle, device, OS, and network conditions used for evaluation.